# Complete Master Guide

Converted from the supplied TXT source; content preserved.

CSE 4165 / CSE 465 WEB PROGRAMMING

FINAL EXAM MASTER GUIDE AND COMPLETE SOLUTIONS (FALL 2023 - SPRING 2026)

PURPOSE

This guide consolidates the uploaded exam papers, the preparation guide, and the available Lab 6, Lab 7, Lab 9, Lab 10 and Lab 11 files. It follows the simple lab style: separate HTML/JavaScript files, standard functions, if/else ladders, PHP forms, explicit MySQLi connection/query/fetch loops, and straightforward SQL. Lab 8 was not included among the uploaded ZIP files.

IMPORTANT SOURCE LIMITS

- The supplied master-guide Markdown stopped after Part 3.4. Its references to an included cheat sheet and a complete Solutions folder were not backed by actual supplied files. This deliverable creates those missing outputs.

- The eight supplied final papers are Fall 2023, Spring 2024, Summer 2024, Fall 2024, Spring 2025, Summer 2025, Fall 2025 and Spring 2026 Set A.

- The Fall 2023 employee table was readable in the supplied PDF, so the practice rows used in the earlier draft have been replaced by the actual eight records from the paper.

- Where a question contradicts its sample output, the conflict is marked. Do not hide such assumptions during exam preparation.

- PHP syntax is lint-checked. JavaScript is syntax-checked and selected sample cases are smoke-tested. A MariaDB/MySQL server was not available here, so database queries still need to be run in XAMPP/phpMyAdmin.

## CHECKPOINT BOARD

# Checkpoint Status

1 Pattern of question making and six question families DONE - rechecked against 8 papers

2 Emphasis chart 2023-2026 DONE - 250 total marks; JS 70, PHP 100, PHP/MySQL 80

3 Most-to-least repeated JS/PHP/SQL syntax DONE - ranked by question coverage; avoid false precision in raw token counts

4 Deduplicated syntax list and syntax chart DONE - see Parts 4-5

5 Lessons, diagrams and examples DONE - see Part 6

6 Exam cheat sheet DONE - see Part 7

7 Solutions to all 25 question groups DONE - see Part 9 and the separate Solutions folder

8 Contradictions, bugs and missing data DONE - see Part 8

9 Expected outputs and sample checks DONE WITH LIMIT - arithmetic/code checks done; SQL runtime not executed

10 Semester/question folders and ZIP DONE - folder tree is included in the deliverables

## PART 0. THE EXAM PATTERN

0.1 Paper map

- Fall 2023: Q1 JavaScript palindrome; Q2 PHP multiplication table; Q3 PHP/MySQL employee table and salary tasks. 30 marks.

- Spring 2024: Q1 JavaScript BMI; Q2 PHP cryptic sequence; Q3 PHP/MySQL electricity bill app. 30 marks.

- Summer 2024: Q1 JavaScript electricity bill; Q2 PHP average-speed function; Q3 PHP password strength; Q4 MySQL sales analysis through PHP. 40 marks.

- Fall 2024: Q1 JavaScript guessing game; Q2 PHP pizza calculator; Q3 PHP/MySQL student grades. 30 marks.

- Spring 2025: Q1 JavaScript password-strength scorer; Q2 PHP venue budget; Q3 PHP/MySQL employee performance. 30 marks.

- Summer 2025: Q1 JavaScript calorie tracker; Q2 PHP screen/seat waste function; Q3 PHP/MySQL sales data. 30 marks.

- Fall 2025: Q1 PHP CT marks; Q2 PHP sales target; Q3 PHP/MySQL library loans. 30 marks.

- Spring 2026 Set A: Q1 JavaScript study tracker; Q2 PHP credits and target; Q3 PHP/MySQL tourism. 30 marks.

Most papers are two hours and 30 marks (three groups of 10). Summer 2024 is the exception: 40 marks and four groups of 10. Seven of the eight papers have a JavaScript question. Every paper contains a database question.

0.2 Six recurring question families

A. JavaScript stateful tracker/game: persistent variables outside functions, running total/counter, range feedback, and a late warning rule. Examples: guessing game, password score, calories, study hours.

B. JavaScript one-shot calculator/string check: read values, convert/check, calculate, output to a label or alert. Examples: palindrome, BMI, electricity bill.

C. PHP whole-unit / leftover / waste function: calculate whole venues/screens/pizzas with ceil(), calculate unused capacity, then waste.

D. PHP classify-and-compare: multiply two values, classify with an if/elseif ladder, and compare total against target.

E. PHP loop/string logic: nested for loops, digit arithmetic with %, password checks with strlen()/substr()/preg_match().

F. PHP + MySQL reports and updates: create a dataset, make a table, insert given rows, SELECT/report, UPDATE rules, and grouped output. Recent versions have four tasks.

0.3 Typical structure

The most common recent layout is Q1 programming, Q2 PHP and Q3 PHP/MySQL, 10 marks each. Summer 2024 adds a second PHP task and a fourth database task. Fall 2025 is the only supplied paper without JavaScript; its first two questions are PHP.

0.4 SQL four-task skeleton

1. SELECT and summarize the initial data (COUNT, SUM, AVG, sometimes GROUP BY/HAVING).

2. UPDATE rows matching a condition.

3. UPDATE a bonus/percentage, often with a cap on the new value.

4. SELECT a grouped report, often ORDER BY ... DESC.

Always read the paper's exact order and conditions. Output for Task 1 is usually best captured before updates; later reports should reflect the updates already executed.

0.5 Forecast (not a guarantee)

- JavaScript: a stateful tracker with a global counter, range messages and an after-N warning is a strong candidate. Still practise a one-shot calculator and string check.

- PHP: classify-and-compare or the ceil/leftover/waste calculation is worth high priority. Also practise loops/digits and password rules.

- SQL: practise full database setup, SELECT, GROUP BY, conditional UPDATE, ORDER BY DESC, HAVING and "only if the result is at most X".

## PART 1. EMPHASIS CHART, 2023-2026

Paper JavaScript PHP PHP + MySQL Total

Fall 2023 10 10 10 30

Spring 2024 10 10 10 30

Summer 2024 10 20 10 40

Fall 2024 10 10 10 30

Spring 2025 10 10 10 30

Summer 2025 10 10 10 30

Fall 2025 0 20 10 30

Spring 2026 10 10 10 30

TOTAL 70 100 80 250

SHARE 28% 40% 32% 100%

Interpretation: PHP accounts for the largest share (40%). The database task is a regular 10-mark component in all 8 papers. JavaScript accounts for 28% overall and appears in 7/8 papers.

Skill / topic Papers out of 8

HTML input + JS DOM read/write 7

JS stateful counter / cumulative data 4

JS array push/join 1

PHP form + POST handling 8 papers contain PHP work; around 10 form-driven question groups

PHP function explicitly requested 4

ceil() whole-unit calculation 3

loop/digit logic 2

string/regular-expression checks 3

Create database/table/insert 8

GROUP BY with COUNT/SUM/AVG 6

Conditional UPDATE 6

ORDER BY DESC / LIMIT 5

HAVING 3

Multi-file HTML -> PHP DB lookup 1

These counts track whether a topic is part of each paper's solution, not a count of how many times each word appears in source code. Use them to set study priority, not as a promise of the next paper.

## PART 2. MOST-TO-LEAST REPEATED SYNTAX

2.1 JavaScript: rank by questions where the pattern is needed

1. document.getElementById("id") - 7/7 JS questions; select input/output elements.

2. .value - 7/7; read a text/select input.

3. function name() and HTML onclick="name()" - 7/7; connect button to code.

4. let variables and if/else comparison logic - 7/7; calculations and decisions.

5. .innerText / .innerHTML - essentially every JS task; display feedback.

6. Number(...) - 5/7; convert numeric input text before calculations.

7. global variable outside function - 4/7; remember state between clicks.

8. += and ++ - common in trackers/scores/counters (4/7).

9. && / || - validation or combined conditions (about 5/7).

10. .toFixed(2) - formatted results (3/7).

11. .length / for / charAt / toLowerCase - string tasks.

12. Math.floor(Math.random()*(max-min+1))+min - random integer game.

13. /[A-Z]/.test(...) and other regex - password categories.

14. array.push(...) and array.join(", ") - collect/list subjects.

15. alert(...), .checked, isNaN(...), .disabled = true - specific prompt requirements.

2.2 PHP: rank by recurrence in solution structure

1. <?php ... ?>, $variables, echo - used in every PHP solution.

2. if / elseif / else - classification, validation and status.

3. <form method="post"> + name="..." + $_POST["..."] - form-based questions.

4. isset($_POST["btn"]) - only calculate after submission.

5. (int) / (float) conversion - numeric form inputs.

6. . string concatenation - building displayed output.

7. function name($a, $b) { ... return ...; } - questions explicitly asking for a reusable calculation.

8. ceil(), min(), max(), abs(), round(), number_format() - round-up, best marks, differences and money formatting.

9. for / while / % / (int)($n / 10) - tables and digit sequence.

10. strlen(), substr(), preg_match() - string/password checks.

11. new mysqli(...), connect_error, query(...), fetch_assoc(), $row["alias"], close() - core database question pattern.

12. array() and associative array keys - the Summer 2025 category-average comparison.

13. echo "<table...>" with <tr>, <td>, <th> - when output is requested as a table.

2.3 SQL: rank by question coverage

1. CREATE DATABASE / USE / CREATE TABLE / INSERT INTO - setup files for 8/8 database questions.

2. SELECT ... FROM - 8/8.

3. WHERE - 7/8.

4. AS aliases - 7/8; make PHP row keys stable and short.

5. UPDATE ... SET - 6/8.

6. AND / OR and comparisons - about 6/8.

7. GROUP BY with COUNT/SUM/AVG - 6/8.

8. ORDER BY ... DESC - 5/8; sort highest first.

9. arithmetic in SET/WHERE (Salary + 5000, Revenue * 1.10) - conditional bonuses/raises.

10. HAVING - 3/8; filter grouped results.

11. LIMIT - useful in the top-two-days and top-N pattern.

12. BETWEEN, LIKE '%word%', COUNT(DISTINCT ...), CASE WHEN - situation-specific but important.

Warning: this ranking counts question-level use, not raw text occurrences. That is more reproducible than pretending an exact count of tokens across different code styles.

## PART 3. DEDUPLICATED SYNTAX INVENTORY

3.1 HTML

- <!DOCTYPE html>, <html lang="en">, <head>, <meta charset="UTF-8">, <title>, <body>.

- <input type="text|number|password">, id="...", name="...", value="...".

- <select id/name> with <option value="...">.

- <input type="radio" name="sameGroup"> and .checked in JS.

- <button onclick="functionName()"> for JavaScript.

- <form method="post"> and <button name="btn"> for PHP.

- <label>, <p>, <br>, <table>, <tr>, <th>, <td>.

- <script src="script.js"></script>.

3.2 JavaScript (duplicates removed)

- let variable = value;

- function name() { ... }

- document.getElementById("id").value

- document.getElementById("id").innerText = output

- document.getElementById("id").innerHTML = output + "<br>" + more

- Number(text), isNaN(value)

- if (...) { } else if (...) { } else { }

- ==, !=, >, <, >=, <=; &&; ||; !

- total += value; count++;

- global state declarations before the function

- for (let i = 0; i < n; i++)

- string.length, string.charAt(i), string.toLowerCase()

- /[A-Z]/.test(text), /[0-9]/.test(text)
- Math.floor(Math.random() * (max - min + 1)) + min
- Math.floor(...), Math.round(...), value.toFixed(2)
- array = []; array.push(item); array.join(", ")
- element.checked, element.disabled = true
- alert("message"); return;

3.3 PHP (duplicates removed)

- <?php ... ?>
- $variable = value; echo ...; . (string concatenation)
- <form method="post">, name="field", $_POST["field"]
- if (isset($_POST["btn"])) { ... }
- (int)$_POST["x"], (float)$_POST["x"]
- if / elseif / else
- function calculate($a, $b) { ... return $result; }
- for ($i=...; ...; ...), while (...)
- $digit = $temp % 10; $temp = (int)($temp / 10)
- ceil(...), min(...), max(...), abs(...), round(...)
- strlen($text), substr($text, -1), preg_match("/[A-Z]/", $text)
- number_format($money, 2)
- $array = array(); $array[$key] = $value
- new mysqli("localhost", "root", "", "database")
- $conn->connect_error; die(...); $conn->query($sql); $res->fetch_assoc(); $conn->close()
- while ($row = $res->fetch_assoc()) { echo $row["alias"]; }
- HTML table output using echo.

3.4 SQL (duplicates removed)

- CREATE DATABASE IF NOT EXISTS name;
- USE name;
- DROP TABLE IF EXISTS table_name;
- CREATE TABLE (... INT, VARCHAR(n), DATE, DECIMAL(p,s), PRIMARY KEY ...);
- INSERT INTO table (...) VALUES (...), (...);
- SELECT columns FROM table;
- WHERE condition; AND; OR; BETWEEN a AND b; LIKE '%text%'; <>; >=; <=.
- AS short_alias.
- COUNT(*), COUNT(DISTINCT col), SUM(col), AVG(col).
- GROUP BY column; HAVING aggregate_condition.
- ORDER BY column DESC; LIMIT number.
- UPDATE table SET col = value WHERE condition.
- UPDATE table SET col = col + amount WHERE ... AND col + amount <= cap.
- SET col = CASE WHEN condition THEN expression ELSE expression END.
- SUM(price * quantity) for a computed total.

## PART 3.5. EXTRA SYNTAX FOUND IN THE LAB FILES

This section is separate from the past-final frequency ranking. These items appear in the provided Lab 6, Lab 7, Lab 9, Lab 10 or Lab 11 material; they are useful revision items but did not form a repeated requirement across the eight past finals.

## JavaScript lab extras:

- console.log(value): print a value in the browser console while debugging.

- document.write("text"): write into the page during initial load; for click-by-click result display, prefer an existing element's innerText/innerHTML.

- alert("message"), confirm("question"), prompt("input message"): popup message, confirmation and input.

- array.pop(), array.push(item): remove/add the final array item.

- array.shift(), array.unshift(item): remove/add the first array item.

- array.splice(index, deleteCount, item): remove or insert at an array position.

- array.sort().reverse(): sort and then reverse an array.

- text.split(" "), text.includes("word"), text.replace(old, next): split/check/replace text.

- element.style.color = "red"; element.style.fontSize = "larger": change an element's appearance.

- document.body.style.backgroundColor = "black": change the page background.

- Toggle pattern: keep `let toggle = 1` outside the function, flip the page style in if/else, and then change toggle to 0/1.

## PHP Lab 9/10 extras:

- <?php ... ?>: PHP code runs on the server; echo prints output into the response page.

- PHP variables begin with $, are case-sensitive, and do not need a declared type. Variable names cannot begin with a number.

- $_GET["field"]: data from URL query parameters; visible in the address bar. $_POST["field"]: data sent in the request body; all form-based past-paper solutions use POST.

- == compares values; === compares value and type. Use break in a switch case to prevent accidental fall-through.

- strlen($s), strtoupper($s), strtolower($s), trim($s), str_replace($find, $replace, $s): common string operations.

- count($array), array_push($array, $item), array_pop($array), in_array($item, $array), sort($array), rsort($array), implode(",", $array): array operations.

- round(), floor(), ceil(), rand($min, $max), max(...), min(...), pow($a, $b), sqrt($x): common math helpers.

- isset($value), is_string($value), is_numeric($value): basic checks.

- class Student { public $name; private $cgpa; ... }: define a class with fields.

- public function __construct($name, $cgpa) { ... }: initialize a new object.

- $this->name: access a field inside the current object.

- public function getCGPA() { return $this->cgpa; }: return an object's value.

- new Student("Name", 3.5): create an object.

- require "carClass.php": load another PHP file.

These OOP constructs are included because they appear in Lab 10, but they are not recurring requirements in the eight supplied final papers. Prioritize the exam pattern first if time is limited.

## STUDY LINKS INCLUDED IN THE PREPARATION-GUIDE PDF (not independently verified):

- Class 1, how to solve final questions: https://youtu.be/tAA6MPUpsvQ

- Class 2, MySQL: https://youtu.be/8y7FIU9AEqg

- Class 3, PHP: https://youtu.be/4_blADh8z9M

- Class 4, JavaScript: https://youtu.be/txUbzXdT4CM

- Final questions repository: https://github.com/mahfuzhasanreza/uiu-web-programming/tree/main/final-questions

- Spring 2025 solutions: https://github.com/mahfuzhasanreza/uiu-web-programming/tree/main/spring-25-final-solution

- Fall 2024 solutions: https://github.com/mahfuzhasanreza/uiu-web-programming/tree/main/fall24-final-solution

- Fall 2025 solutions: https://github.com/mahfuzhasanreza/uiu-web-programming/tree/main/fall25-final-solution

- Summer 2025 solutions: https://github.com/mahfuzhasanreza/uiu-web-programming/tree/main/summer-25-final-solution

- Basics for the final: https://github.com/mahfuzhasanreza/uiu-web-programming/blob/main/README.md

## PART 4. SYNTAX CHART: FUNCTION AND WHEN TO APPLY

Syntax / example Function and when to use

getElementById("price").value Read a field; every JS input task.

Number(value) Convert numeric string; before math.

innerText = "..." Display plain feedback text.

innerHTML = "...<br>..." Display output with HTML line breaks.

```
function calculate() + onclick="calculate()"          Run calculation after a button click.
let total = 0 outside function                         Keep accumulated value across clicks.
```

total += amount Add new values to the running total.

count++ Count attempts, sessions or entries.

```
if / else if / else                                   Test ordered ranges or categories.
```

Math.floor(Math.random()*(max-min+1))+min Generate random integer within both limits.

/[A-Z]/.test(password) Check if a password has uppercase text.

array.push(subject), array.join(", ") Save items and print them as a list.

$form method="post" + name="x" Send fields to PHP.

isset($_POST["btn"]) Avoid calculation on a first page visit.

(int) / (float)$_POST["x"] Convert form value before math.

ceil($people / $capacity) Round up to complete venues/screens/pizzas.

min($a,$b,$c) Find lowest CT mark.

abs($difference) Display a positive size for a below-target gap.

strlen / substr / preg_match Length, last character and pattern checks.

new mysqli(...) Connect to the local MySQL/MariaDB database.

$conn->query($sql) Run SQL from PHP.

$res->fetch_assoc() Read one row as an associative array.

$row["alias"] Read a named field returned by SQL.

```
SELECT ... WHERE ...                                  Filter rows before grouping.
```

GROUP BY category Return one aggregate row per category.

HAVING AVG(Rating) > 4.4 Filter groups after aggregate calculation.

ORDER BY total DESC Put highest totals first.

```
UPDATE ... SET ... WHERE ...                          Change only matching rows.
```

AND NewValue <= limit Enforce a cap on the resulting value.

CASE WHEN ... THEN ... ELSE ... END Apply alternative changes in one UPDATE.

COUNT(DISTINCT sale_date) Count distinct days, not sale rows.

SUM(EntryFee * VisitorsPerYear) Calculate region revenue from row-level products.

## PART 5. SEVEN CORE LESSONS WITH DIAGRAMS

### Lesson 1 - JavaScript state persists only when declared outside the click function.

# Page loads:

```
      let total = 0; let count = 0;
```

|

v

Click 1: total = total + value1; count = count + 1

|

v

Click 2: total = previous total + value2; count = count + 1

If total and count are declared inside the function, they reset every click. Trackers require global variables.

## Lesson 2 - JS button/input/output pattern.

HTML input id="hours" -> button onclick="addStudy()" -> JS reads .value

-> Number(...)

-> calculate / if-else

-> output element .innerText

## Lesson 3 - PHP form request pattern.

First visit (GET) -> page displays the HTML form -> isset($_POST["btn"]) is false

Click Submit (POST) -> PHP reads $_POST fields -> casts values -> calculates -> echo result

Remember: JavaScript identifies an input/output with id. PHP receives a form field by name.

## Lesson 4 - SQL clause order.

```
SELECT columns / aggregates
```

FROM table

WHERE row_condition

GROUP BY group_column

HAVING aggregate_condition

ORDER BY sort_column DESC

LIMIT n;

WHERE filters individual rows before grouping. HAVING filters groups after aggregation.

## Lesson 5 - Ceiling and waste.

needed = people * units_per_person

units = ceil(needed / capacity)

leftover = units * capacity - needed

waste = leftover * cost_per_seat_or_slice

Example: 10 students * 3 slices = 30 slices. Each pizza holds 8. ceil(30/8)=4 pizzas; capacity=32; leftover=2. At 1050 BDT per pizza, cost per slice=1050/8=131.25; waste=2*131.25=262.50 BDT.

## Lesson 6 - Best two CT marks.

best_two_average = (ct1 + ct2 + ct3 - min(ct1,ct2,ct3)) / 2

Example: CT marks 8, 12, 15 -> (8+12+15-8)/2 = 13.5.

## Lesson 7 - Round-trip average speed.

# If the distance up and down is the same:

distance = uphill_speed * uphill_time

downhill_time = distance / downhill_speed

average_speed = (2 * distance) / (uphill_time + downhill_time)

Do not use the ordinary arithmetic mean (up_speed + down_speed)/2 when the two travel times differ.

### PART 6. EXAM CHEAT SHEET

## JavaScript starter:

```
let total = 0;
let count = 0;
function run() {
    let x = Number(document.getElementById("input").value);
```

total += x;

count++;

```
    if (total <= 5) { message = "..."; }
```

else if (total <= 12) { message = "..."; }

else { message = "..."; }

document.getElementById("out").innerText = message;

}

## PHP form starter:

<form method="post">

Value: <input type="number" name="x">

<button name="btn">Submit</button>

</form>

```
<?php
if (isset($_POST["btn"])) {
```

$x = (float)$_POST["x"];

echo $x;

}

?>

## PHP/MySQL starter:

$conn = new mysqli("localhost", "root", "", "database_name");

```
if ($conn->connect_error) { die("Connection failed: " . $conn->connect_error); }
```

$res = $conn->query("SELECT column_name AS alias FROM table_name");

```
while ($row = $res->fetch_assoc()) { echo $row["alias"] . "<br>"; }
```

$conn->close();

## SQL group starter:

```
SELECT group_col, COUNT(*) AS total
```

FROM table_name

WHERE condition

GROUP BY group_col

HAVING COUNT(*) > 1

ORDER BY total DESC;

## SQL safe cap pattern:

```
UPDATE table_name
```

SET score = score + 5

WHERE score > 80 AND score + 5 <= 90;

LAST-MINUTE TRAPS

- Inputs are strings until converted to numbers.

- Keep running totals and counters outside JS functions.

- The later warning rule may override the ordinary feedback; apply it after the normal if/else ladder.

- Use ceil(), not round(), when complete units must be booked.

- Use WHERE for rows and HAVING for groups.

- If a cap refers to the final value, test the expression for the new value in WHERE.

- Use different IDs for different HTML buttons; the Lab 11 sample has a duplicate id bug.

- Use isset($_POST["btn"]) before reading PHP form values.

- Use SQL aliases (AS total, AS avg_rating) and then read those aliases in PHP.

- Re-import setup.sql before every fresh test because UPDATE statements change the data.

- Do not execute UPDATE statements before a report that must reflect the original dataset.

## PART 7. CONTRADICTIONS, BUGS AND SOURCE LIMITS

1. Fall 2023 Q3 real data: the PDF includes eight rows (SKING through DLORENTZ) with salaries 24000, 17000, 17000, 9000, 6000, 4800, 4800, 4200. Their total is 86800. The earlier draft used invented employee rows; those have been replaced by the actual table. The sample 27.65% requires the original total. The solution therefore shows salary percentages before the salary updates, even though the written subtask order lists the raise first.

2. Fall 2023 Q4: use one CASE UPDATE so a salary does not get changed twice.

3. Spring 2024 Q2: text says add the product of digits; printed sequences match adding the digit sum. The supplied solution follows the printed examples and leaves the product alternative commented.

4. Spring 2024 Q1: boundary at BMI exactly 20 is not specified cleanly. Solution treats 20 through 30 as “not bad.”

5. Spring 2024 Q3: the schema figure is low quality / partially extracted. The solution reconstructs fields as Location, Area, Rate1, Rate2, Rate3, Tax. The stated tier ranges are implemented as progressive slabs. For 100 units in Dhaka Urban: 75*4.19 + 25*5.72 = 457.25; tax 91.45; total 548.70.

6. Summer 2024 Q1: the phrase says surcharge on total bill and rates are location-specific. Apply 5% for bill >=500 and <=1000, 10% for bill >1000, and no surcharge below 500.

7. Summer 2024 Q2: wording mixes mph and km/h; sample values are consistent with the same-distance average-speed calculation. Return an integer as requested.

8. Fall 2024 Q1: examples use secret values 37, 82 and 15 outside the written 500-5000 range. The code follows the written range 500-5000.

9. Spring 2025 Q1: “+10 per 2 characters” has more than one possible interpretation. The lab formula used by the sample examples is floor((length-6)/2)*10 after the minimum 6-character rule.

10. Spring 2025 Q2: statement mentions a fixed 15,000 BDT venue cost, but the sample calculations use empty seats * cost per person (e.g. 70*120=8400). The solution follows samples; do not add 15000 to the wasted-money figure.

11. Fall 2025 Q2: written category rule says Excellent >=500, but sample labels 400 as Excellent. The solution uses >=400 to reproduce the sample; if asked to follow only the prose, change the threshold to 500.

12. Summer 2025 Q2: fixed screen rental of 25,000 BDT is not included in the sample “wasted money”; sample uses empty seats * ticket price. Solution follows the samples.

13. Spring 2026 Q1: use `totalHours < goal` for warning to avoid a fractional gap such as 19.5 hours.

14. Lab bugs carried into the earlier code: Fall CT solution added ct2 twice and did not guard submit; Lab 11 duplicate button id and count boundary were corrected in the solutions.

15. SQL fields from aggregate results should always be aliased. Do not depend on field names like `AVG(Rating)` unless forced.

16. Data state: re-import the setup SQL before testing because query scripts make permanent updates.

## PART 8. EXPECTED OUTPUT / SELF-TEST TABLE

Case / input Expected output

Fall 2023 Q1: 1aBcbA1 Yes. It is a Palindrome.

Fall 2023 Q1: baCbb No. It is not a Palindrome.

Fall 2023 Q1: 79wsasw97 Yes. It is a Palindrome.

Spring 2024 Q2 seed 7, terms 5 7, 14, 19, 29, 40 (digit-sum interpretation)

Spring 2024 Q2 seed 4, terms 9 4, 8, 16, 23, 28, 38, 49, 62, 70 (digit-sum interpretation)

Summer 2024 Q2 calculate(18,20,60) 30

Summer 2024 Q2 calculate(30,8,24) 12

Summer 2024 Q1 Dhaka Urban 100 units Bill 750; surcharge 37.50; final 787.50 BDT

Spring 2024 Q3 Dhaka Urban 100 units (progressive) Bill 457.25; tax 91.45; final 548.70 BDT

Fall 2024 Q2: 10,3,8 4 pizzas, 2 leftovers, 262.50 BDT wasted

Fall 2024 Q2: 7,2,6 3 pizzas, 4 leftovers, 700 BDT wasted

Fall 2024 Q2: 12,2,4 6 pizzas, 0 leftovers, 0 wasted

Spring 2025 Q1: abc score 0, Very Weak

Spring 2025 Q1: hello123 score 45, Weak

Spring 2025 Q1: Strong@Pass score 75, Strong

Spring 2025 Q1: Perfect@Pass123 score 115, Perfect Password

Spring 2025 Q2: 250 attendees, cost 120, capacity 80 4 venues, 70 empty, 8400 BDT waste

Summer 2025 Q2: 150, capacity 60, ticket 500 3 screens, 30 empty, 15000 BDT waste

Fall 2025 Q2: 25 per day*12 days vs target 300 300, Good, target exact

Fall 2025 Q2: 40 per day*10 days vs target 350 400, Excellent (sample rule), above by 50

Spring 2026 Q1 sample sessions total 14 / average 3.5 / Almost ready!

Spring 2026 Q1 sample sessions total 21 / average 5.25 / Weekly goal reached!

Spring 2026 Q1: 8 sessions, total 15 hours Increase your study time!

Spring 2026 Q2: 4 credits*30 courses vs 120 target 120, Excellent, exact target

Summer 2024 Q4 product totals product 103: qty 5/revenue 960; product 107: qty 15/revenue 2670

Fall 2024 Q3 initial grade counts A:2, B:2, C:1, D:1

Spring 2025 Q3 bonus Marium salary 52000 -> 57000

Summer 2025 Q3 updated revenue Laptop 385000; Desk 79200; Bottle 30000; Pen 20000

Fall 2025 Q3 fee updates Jabbar 24 -> 26.40; Karim 30 -> 33; eligible fees only

Spring 2026 Q3 region revenue after task 2 KHL 13,200,000; SYL 6,000,000; BGH 2,100,000; NWG 1,320,000

## PART 9. SOLUTIONS FOLDER MAP

Fall_2023/

Q1_JS_Palindrome/ (index.html, script.js)

Q2_PHP_Multiplication_Table/ (q2.php)

Q3_PHP_MySQL_Employee/ (setup.sql, q3.php)

Spring_2024/

Q1_JS_BMI/ (index.html, script.js)

Q2_PHP_Cryptic_Sequence/ (q2.php)

Q3_PHP_MySQL_Electricity_Bill/ (setup.sql, index.html, bill.php)

Summer_2024/

Q1_JS_Electricity_Bill/ (index.html, script.js)

Q2_PHP_Average_Speed/ (q2.php)

Q3_PHP_Password_Strength/ (q3.php)

Q4_MySQL_Sales_Analysis/ (setup.sql, q4.php)

Fall_2024/

Q1_JS_Number_Guessing_Game/ (index.html, script.js)

Q2_PHP_Pizza_Calculator/ (q2.php)

Q3_PHP_MySQL_Student_Grades/ (setup.sql, q3.php)

Spring_2025/

Q1_JS_Password_Strength_Scorer/ (index.html, script.js)

Q2_PHP_Venue_Budget/ (q2.php)

Q3_PHP_MySQL_Employee_Performance/ (setup.sql, q3.php)

Summer_2025/

Q1_JS_Calorie_Tracker/ (index.html, script.js)

Q2_PHP_Movie_Screens/ (q2.php)

Q3_PHP_MySQL_Sales_Data/ (setup.sql, q3.php)

Fall_2025/

Q1_PHP_CT_Marks/ (q1.php)

Q2_PHP_Sales_Target/ (q2.php)

Q3_PHP_MySQL_Library_Loans/ (setup.sql, q3.php)

Spring_2026/

Q1_JS_Study_Tracker/ (index.html, script.js)

Q2_PHP_Academic_Credits/ (q2.php)

Q3_PHP_MySQL_Tourism/ (setup.sql, q3.php)

## Run locally:

1. Install/start XAMPP. Start Apache and MySQL.

2. Copy a semester folder under C:\xampp\htdocs\.

3. Import the matching setup.sql in phpMyAdmin before opening q3.php or q4.php.

4. Open the PHP file through http://localhost/...; do not double-click a PHP file as a local file.

5. JavaScript tasks can be opened directly via index.html.

6. Re-import setup.sql before each practice attempt to restore initial data.

END OF STUDY GUIDE BODY. FULL QUESTION SOLUTION SOURCE LISTINGS FOLLOW.

==========

Fall_2023 / Q1_JS_Palindrome

==========

--- FILE: index.html ---

<!DOCTYPE html>

```
  <html lang="en">
```

<head><meta charset="UTF-8"><title>Palindrome Checker</title></head>

<body>

Enter an alphanumeric string: <input type="text" id="text">

<button onclick="check()">Check</button>

<label id="ans"></label>

<script src="script.js"></script>

</body>

</html>

--- FILE: script.js ---

```
   function check() {
        let text = document.getElementById("text").value.toLowerCase();
        let isPal = true;

        for (let i = 0; i < text.length / 2; i++) {
            if (text.charAt(i) != text.charAt(text.length - 1 - i)) {
```

isPal = false;

}

}

```
        if (isPal) {
```

document.getElementById("ans").innerText = "Yes. It is a Palindrome.";

} else {

document.getElementById("ans").innerText = "No. It is not a Palindrome.";

}

}

==========

Fall_2023 / Q2_PHP_Multiplication_Table

==========

--- FILE: q2.php ---

<!DOCTYPE html>

```
   <html lang="en">
```

<head><meta charset="UTF-8"><title>Multiplication Table</title></head>

<body>

<form method="post">

Enter the value of n: <input type="number" name="n" value="10">

<button name="btn">Generate Table</button>

</form>

```
        <?php
        if (isset($_POST["btn"])) {
```

$n = (int)$_POST["n"];

```
            if ($n > 0) {
```

echo "<table border='1' cellpadding='5'>";

```
                for ($i = 1; $i <= $n; $i++) {
```

echo "<tr>";

```
                    for ($j = 1; $j <= $n; $j++) {
```

echo "<td>" . ($i * $j) . "</td>";

}

echo "</tr>";

}

echo "</table>";

} else {

echo "Enter a positive value for n.";

}

}

```
?>
</body>
</html>

================================================================================================
Fall_2023 / Q3_PHP_MySQL_Employee
================================================================================================

--- FILE: setup.sql ---
   CREATE DATABASE IF NOT EXISTS company;
USE company;
DROP TABLE IF EXISTS employee;
   CREATE TABLE employee (
EMAIL VARCHAR(20) PRIMARY KEY,
PHONE_NUMBER VARCHAR(20),
HIRE_DATE DATE NOT NULL,
JOB_ID VARCHAR(20) NOT NULL,
SALARY DECIMAL(10,2) NOT NULL
);
INSERT INTO employee VALUES
('SKING','515.123.4567','1987-06-17','AD_PRES',24000.00),
('NKOCHHAR','515.123.4568','1987-06-18','AD_VP',17000.00),
('LDEHAAN','515.123.4569','1987-06-19','AD_VP',17000.00),
('AHUNOLD','590.423.4567','1987-06-20','IT_PROG',9000.00),
('BERNST','590.423.4568','1987-06-21','IT_PROG',6000.00),
('DAUSTIN','590.423.4569','1987-06-22','IT_PROG',4800.00),
('VPATABAL','590.423.4560','1987-06-23','IT_PROG',4800.00),
('DLORENTZ','590.423.5567','1987-06-24','IT_PROG',4200.00);

--- FILE: q3.php ---
   <?php
$conn = new mysqli("localhost", "root", "", "company");
   if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}

// Task 3 sample says 24000 / original total 86800 = 27.65%.
// Therefore, show percentages before changing the salaries.
$res = $conn->query("SELECT SUM(SALARY) AS total_salary FROM employee");
$row = $res->fetch_assoc();
$total = $row["total_salary"];

echo "<h3>Salary Percentage (original salaries)</h3>";
$res = $conn->query("SELECT EMAIL, SALARY FROM employee");
echo "<table border='1'><tr><th>EMAIL</th><th>PERCENTAGE</th></tr>";
   while ($row = $res->fetch_assoc()) {
$percent = $row["SALARY"] / $total * 100;
echo "<tr><td>" . $row["EMAIL"] . "</td><td>" . number_format($percent, 2) . "%</td></tr>";
}
```

```
echo "</table>";

// Task 2: most recent gets 17.5%; most senior gets 21.5%.
$conn->query("UPDATE employee SET SALARY = SALARY * 1.175 ORDER BY HIRE_DATE DESC LIMIT 1");
$conn->query("UPDATE employee SET SALARY = SALARY * 1.215 ORDER BY HIRE_DATE ASC LIMIT 1");

// Task 4: one CASE statement prevents a row from being updated twice.
$conn->query("UPDATE employee
SET SALARY = CASE WHEN SALARY > 10000 THEN SALARY * 0.90
ELSE SALARY * 1.10 END");

echo "<p>Salary updates completed.</p>";
$conn->close();
?>
```

==========

Spring_2024 / Q1_JS_BMI

==========

--- FILE: index.html ---

```
<!DOCTYPE html>
   <html lang="en">
<head><meta charset="UTF-8"><title>BMI Calculator</title></head>
<body>
Height (cm): <input type="text" id="cm"><br>
Weight (lb): <input type="text" id="lb"><br>
<button onclick="calc()">Calculate</button>
<p id="res"></p>
<script src="script.js"></script>
</body>
</html>
```

--- FILE: script.js ---

```
   function calc() {
        let cm = Number(document.getElementById("cm").value);
        let lb = Number(document.getElementById("lb").value);
        if (cm <= 0 || lb <= 0 || isNaN(cm) || isNaN(lb)) {
document.getElementById("res").innerText = "Enter positive numbers.";
return;
}
        let metre = cm / 100;
        let kg = lb / 2.2046;
        let result = kg / (metre * metre);
        let status;
        if (result < 20) {
status = "bad";
} else if (result <= 30) {
status = "not bad";
} else {
status = "good";
}
```

```
document.getElementById("res").innerHTML =
"Result: " + result.toFixed(2) + "<br>Status: " + status;
}
```

==========

Spring_2024 / Q2_PHP_Cryptic_Sequence

==========

--- FILE: q2.php ---

```
<form method="post">
Seed value: <input type="number" name="seed"><br>
Terms: <input type="number" name="terms"><br>
<button name="btn">Generate</button>
</form>
   <?php
   if (isset($_POST["btn"])) {
$n = (int)$_POST["seed"];
$terms = (int)$_POST["terms"];
       for ($i = 1; $i <= $terms; $i++) {
echo $n;
           if ($i < $terms) {
echo ", ";
}

$temp = $n;
$digitSum = 0;
$digitProduct = 1;
           while ($temp > 0) {
$digit = $temp % 10;
$digitSum = $digitSum + $digit;
$digitProduct = $digitProduct * $digit;
$temp = (int)($temp / 10);
}

// The written rule says product, but the printed samples follow digit sum.
// Keep digitSum to match the sample outputs. Use digitProduct for the written rule.
$n = $n + $digitSum;
// $n = $n + $digitProduct;
}
}
?>
```

==========

Spring_2024 / Q3_PHP_MySQL_Electricity_Bill

==========

--- FILE: setup.sql ---

```
   CREATE DATABASE IF NOT EXISTS electricity_bill;
USE electricity_bill;
```

```
DROP TABLE IF EXISTS bill_info;
   CREATE TABLE bill_info (
id INT AUTO_INCREMENT PRIMARY KEY,
Location VARCHAR(30) NOT NULL,
Area VARCHAR(10) NOT NULL,
Rate1 DECIMAL(6,2) NOT NULL,
Rate2 DECIMAL(6,2) NOT NULL,
Rate3 DECIMAL(6,2) NOT NULL,
Tax INT NOT NULL
);
INSERT INTO bill_info (Location, Area, Rate1, Rate2, Rate3, Tax) VALUES
('Dhaka','Urban',4.19,5.72,6.00,20),
('Dhaka','Rural',3.19,4.72,5.00,20),
('Chittagong','Urban',3.50,4.80,5.23,15),
('Chittagong','Rural',2.50,3.80,4.23,15);

--- FILE: index.html ---
<!DOCTYPE html>
   <html lang="en">
<head><meta charset="UTF-8"><title>Electricity Bill</title></head>
<body>
<form method="post" action="bill.php">
```

## Location:

```
<select name="location">
<option value="Dhaka">Dhaka</option>
<option value="Chittagong">Chittagong</option>
</select><br>
```

## Area:

```
<input type="radio" name="area" value="Urban" checked> Urban
<input type="radio" name="area" value="Rural"> Rural<br>
Units consumed: <input type="number" name="units" min="0.01" step="any"><br>
<button name="btn">Calculate</button>
</form>
</body>
</html>

--- FILE: bill.php ---
   <?php
   if (isset($_POST["btn"])) {
$location = $_POST["location"];
$area = $_POST["area"];
$units = (float)$_POST["units"];

$conn = new mysqli("localhost", "root", "", "electricity_bill");
       if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}
```

```
$location = $conn->real_escape_string($location);
$area = $conn->real_escape_string($area);
$res = $conn->query("SELECT * FROM bill_info WHERE Location='$location' AND Area='$area'");
       if ($units <= 0 || !$res || $res->num_rows == 0) {
echo "Enter valid information.";
} else {
$row = $res->fetch_assoc();

// Progressive slabs: first 75 units, next 125 units, then remaining units.
           if ($units <= 75) {
$bill = $units * $row["Rate1"];
} elseif ($units <= 200) {
$bill = 75 * $row["Rate1"] + ($units - 75) * $row["Rate2"];
} else {
$bill = 75 * $row["Rate1"] + 125 * $row["Rate2"] + ($units - 200) * $row["Rate3"];
}

$tax = $bill * $row["Tax"] / 100;
echo "Bill: " . number_format($bill, 2) . " BDT<br>";
echo "Tax: " . number_format($tax, 2) . " BDT<br>";
echo "Total: " . number_format($bill + $tax, 2) . " BDT";
}
$conn->close();
}
?>
```

```
==========================================================================================
Summer_2024 / Q1_JS_Electricity_Bill
==========================================================================================

--- FILE: index.html ---
<!DOCTYPE html>
   <html lang="en">
<head><meta charset="UTF-8"><title>Electricity Bill</title></head>
<body>
```

# Location:

```
<select id="location">
<option value="">Select</option>
<option value="Dhaka">Dhaka</option>
<option value="Chattogram">Chattogram</option>
<option value="Other">Other</option>
</select><br>
```

# Area:

```
<input type="radio" id="urban" name="area"> Urban
<input type="radio" id="rural" name="area"> Rural<br>
```

Unit Consumed: <input type="text" id="units"><br>

<button onclick="calc()">Calculate</button>

<script src="script.js"></script>

</body>

</html>

--- FILE: script.js ---

```
function calc() {
    let location = document.getElementById("location").value;
    let urban = document.getElementById("urban").checked;
    let rural = document.getElementById("rural").checked;
    let raw = document.getElementById("units").value;
    let units = Number(raw);
    if (location == "") {
```

alert("Select a location.");

return;

}

```
    if (!urban && !rural) {
```

alert("Choose Urban or Rural.");

return;

}

```
    if (raw == "" || isNaN(units) || units <= 0) {
```

alert("Unit Consumed must be a positive number.");

return;

}

```
    let rate;
    if (location == "Dhaka") {
```

rate = urban ? 7.5 : 6.5;

} else if (location == "Chattogram") {

rate = urban ? 7.0 : 6.0;

} else {

rate = urban ? 6.5 : 5.5;

}

```
    let bill = units * rate;
    let surcharge = 0;
    if (bill > 1000) {
```

surcharge = bill * 0.10;

} else if (bill >= 500) {

surcharge = bill * 0.05;

}

alert("Final bill: " + (bill + surcharge).toFixed(2) + " taka");

}

==========

Summer_2024 / Q2_PHP_Average_Speed

==========

--- FILE: q2.php ---

```
<?php
function calculate($minutes, $up, $down) {
```

$hoursUp = $minutes / 60;

```
$distance = $up * $hoursUp;
$hoursDown = $distance / $down;
$average = (2 * $distance) / ($hoursUp + $hoursDown);
return (int)round($average);
}

echo calculate(18, 20, 60) . "<br>"; // 30
echo calculate(30, 8, 24); // 12
?>
```

==========

Summer_2024 / Q3_PHP_Password_Strength

==========

--- FILE: q3.php ---

```
<form method="post">
Password: <input type="password" name="pw">
<button name="btn">Check</button>
</form>
   <?php
   if (isset($_POST["btn"])) {
$password = $_POST["pw"];
$length = strlen($password);
$upper = preg_match("/[A-Z]/", $password);
$lower = preg_match("/[a-z]/", $password);
$digit = preg_match("/[0-9]/", $password);
$special = preg_match("/[^A-Za-z0-9]/", $password);
       if ($length >= 10 && $upper && $lower && $digit && $special && substr($password, -1) == "*") {
$strength = "Strong";
} elseif ($length >= 8 && $upper && $lower && $digit) {
$strength = "Medium";
} elseif ($length >= 6 && $upper) {
$strength = "Poor";
} else {
$strength = "Very Poor";
}
echo $strength;
}
?>
```

==========

Summer_2024 / Q4_MySQL_Sales_Analysis

==========

--- FILE: setup.sql ---

```
   CREATE DATABASE IF NOT EXISTS retail;
USE retail;
DROP TABLE IF EXISTS sales;
   CREATE TABLE sales (
```

```
sale_id INT PRIMARY KEY,
product_id INT NOT NULL,
sale_date DATE NOT NULL,
quantity_sold INT NOT NULL,
total_price DECIMAL(10,2) NOT NULL
);
INSERT INTO sales VALUES
(1,107,'2024-01-01',5,2500.00),
(2,103,'2024-01-02',3,900.00),
(3,103,'2024-01-02',2,60.00),
(4,107,'2024-01-03',4,80.00),
(5,107,'2024-01-03',6,90.00);

--- FILE: q4.php ---
   <?php
$conn = new mysqli("localhost", "root", "", "retail");
   if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}

// Task 1: total quantity and total revenue for each product.
echo "<h3>1. Product totals</h3>";
$res = $conn->query("SELECT product_id, SUM(quantity_sold) AS qty, SUM(total_price) AS revenue
FROM sales GROUP BY product_id");
   while ($row = $res->fetch_assoc()) {
echo $row["product_id"] . " | Quantity: " . $row["qty"] . " | Revenue: " . $row["revenue"] . "<br>";
}

// Task 2: two days with the highest revenue in January 2024.
echo "<h3>2. Top two days</h3>";
$res = $conn->query("SELECT sale_date, SUM(total_price) AS revenue
FROM sales
WHERE sale_date BETWEEN '2024-01-01' AND '2024-01-31'
GROUP BY sale_date ORDER BY revenue DESC LIMIT 2");
   while ($row = $res->fetch_assoc()) {
echo $row["sale_date"] . " | " . $row["revenue"] . "<br>";
}

// Task 3: average quantity only where sales took place on 2+ distinct days.
echo "<h3>3. Average quantity</h3>";
$res = $conn->query("SELECT product_id, AVG(quantity_sold) AS avg_qty
FROM sales
WHERE sale_date BETWEEN '2024-01-01' AND '2024-01-31'
GROUP BY product_id
HAVING COUNT(DISTINCT sale_date) >= 2");
   while ($row = $res->fetch_assoc()) {
echo $row["product_id"] . " | " . $row["avg_qty"] . "<br>";
}
$conn->close();
```

?>

==========

Fall_2024 / Q1_JS_Number_Guessing_Game

==========

--- FILE: index.html ---

```
<!DOCTYPE html>
  <html lang="en">
<head><meta charset="UTF-8"><title>Number Guessing Game</title></head>
<body>
<input type="text" id="guess">
<button id="submitBtn" onclick="numberGuess()">Submit</button>
<button id="resetBtn" onclick="resetGame()">Reset</button><br>
<label id="ans"></label>
<script src="script.js"></script>
</body>
</html>
```

--- FILE: script.js ---

```
  let secret = Math.floor(Math.random() * (5000 - 500 + 1)) + 500;
  let count = 0;
  function numberGuess() {
count++;
      let guess = Number(document.getElementById("guess").value);
      let ans = document.getElementById("ans");
      if (guess == secret) {
ans.innerText = "Correct!";
document.getElementById("guess").disabled = true;
document.getElementById("submitBtn").disabled = true;
} else {
          if (guess > secret) {
ans.innerText = "Too high!";
} else {
ans.innerText = "Too low!";
}
          if (count >= 5) {
ans.innerText += " Out of guesses!";
document.getElementById("guess").disabled = true;
document.getElementById("submitBtn").disabled = true;
}
}
}
  function resetGame() {
secret = Math.floor(Math.random() * (5000 - 500 + 1)) + 500;
count = 0;
document.getElementById("guess").disabled = false;
document.getElementById("submitBtn").disabled = false;
document.getElementById("guess").value = "";
```

```
document.getElementById("ans").innerText = "";
}
```

==========

Fall_2024 / Q2_PHP_Pizza_Calculator

==========

--- FILE: q2.php ---

```
<form method="post">
Students: <input type="number" name="students"><br>
Slices per student: <input type="number" name="perStudent"><br>
Slices per pizza: <input type="number" name="perPizza"><br>
<button name="btn">Calculate</button>
</form>
   <?php
   function calculate($students, $perStudent, $perPizza) {
       if ($perPizza <= 0) {
echo "Slices per pizza must be greater than zero.";
return;
}

$needed = $students * $perStudent;
$pizzas = ceil($needed / $perPizza);
$left = $pizzas * $perPizza - $needed;
$wasted = (1050 / $perPizza) * $left;

echo "<table border='1'><tr><th>Total Pizzas</th><th>Leftover Slices</th><th>Wasted Money (BDT)</th></tr>";
echo "<tr><td>$pizzas</td><td>$left</td><td>" . number_format($wasted, 2) . "</td></tr></table>";
}
   if (isset($_POST["btn"])) {
calculate((int)$_POST["students"], (int)$_POST["perStudent"], (int)$_POST["perPizza"]);
}
?>
```

==========

Fall_2024 / Q3_PHP_MySQL_Student_Grades

==========

--- FILE: setup.sql ---

```
   CREATE DATABASE IF NOT EXISTS uiuweb_final;
USE uiuweb_final;
DROP TABLE IF EXISTS student_final;
   CREATE TABLE student_final (
StudentID INT PRIMARY KEY,
StudentName VARCHAR(50) NOT NULL,
CourseID INT NOT NULL,
CourseTitle VARCHAR(60) NOT NULL,
Grade INT NOT NULL,
LetterGrade VARCHAR(2) NOT NULL
);
```

```
INSERT INTO student_final VALUES
(1,'Karim Uddin',101,'Web Programming',85,'B'),
(2,'Rahim Ahmed',101,'Web Programming',92,'A'),
(3,'Jashim Hossain',102,'Project Management',78,'C'),
(4,'Jasica Ahmed',101,'Web Programming',65,'D'),
(5,'Faria Karim',102,'Project Management',95,'A'),
(6,'Niassoh Dihan',103,'System Analysis and Design',80,'B');

--- FILE: q3.php ---
   <?php
$conn = new mysqli("localhost", "root", "", "uiuweb_final");
   if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}

echo "<h3>1. Students per letter grade</h3>";
$res = $conn->query("SELECT LetterGrade, COUNT(*) AS total FROM student_final GROUP BY LetterGrade");
   while ($row = $res->fetch_assoc()) {
echo $row["LetterGrade"] . ": " . $row["total"] . "<br>";
}

$conn->query("UPDATE student_final SET LetterGrade='C' WHERE Grade < 75 AND LetterGrade <> 'D'");
$conn->query("UPDATE student_final SET Grade = Grade + 5 WHERE Grade > 80 AND Grade + 5 <= 90");
echo "<p>2. Letter grades updated where required.</p>";
echo "<p>3. Bonus marks updated where the final grade does not exceed 90.</p>";

echo "<h3>4. Students per course</h3>";
$res = $conn->query("SELECT CourseTitle, COUNT(*) AS total FROM student_final
GROUP BY CourseTitle ORDER BY total DESC");
   while ($row = $res->fetch_assoc()) {
echo $row["CourseTitle"] . ": " . $row["total"] . "<br>";
}
$conn->close();
?>

================================================================================================
Spring_2025 / Q1_JS_Password_Strength_Scorer
================================================================================================

--- FILE: index.html ---
<!DOCTYPE html>
   <html lang="en">
<head><meta charset="UTF-8"><title>Password Strength</title></head>
<body>
<input type="password" id="password">
<button onclick="checkStrength()">Check Strength</button><br>
<label id="label"></label>
<script src="script.js"></script>
</body>
</html>
```

--- FILE: script.js ---

```
let attempts = 0;
function checkStrength() {
attempts++;
    let password = document.getElementById("password").value;
    let score = 0;
    if (password.length >= 6) {
score += Math.floor((password.length - 6) / 2) * 10;
        if (/[A-Z]/.test(password)) { score += 15; }
        if (/[a-z]/.test(password)) { score += 15; }
        if (/[0-9]/.test(password)) { score += 20; }
        if (/[!@#$%^&*]/.test(password)) { score += 25; }
}
    let level;
    if (score >= 100) { level = "Perfect Password! Success!"; }
else if (score >= 91) { level = "Very Strong"; }
else if (score >= 71) { level = "Strong"; }
else if (score >= 51) { level = "Medium"; }
else if (score >= 31) { level = "Weak"; }
else { level = "Very Weak"; }
    if (attempts > 8 && score < 71) {
level += "\nNeed practice! Use 10+ characters with upper, lower, digits and !@#$%^&*";
}
document.getElementById("label").innerText =
level + "\nScore: " + score + "\nAttempt: " + attempts;
}
```

==========

Spring_2025 / Q2_PHP_Venue_Budget

==========

--- FILE: q2.php ---

```
<form method="post">
Attendees: <input type="number" name="people"><br>
Cost per person: <input type="number" name="cost"><br>
Venue capacity: <input type="number" name="cap"><br>
<button name="btn">Calculate</button>
</form>
<?php
function calculate($people, $cost, $cap) {
    if ($cap <= 0) {
echo "Venue capacity must be greater than zero.";
return;
}
$venues = ceil($people / $cap);
$empty = $venues * $cap - $people;
$wasted = $empty * $cost;
echo "Total Venues: $venues<br>Empty Seats: $empty<br>";
```

```
echo "Wasted Money (BDT): " . number_format($wasted, 2);
// The paper also mentions a fixed 15,000 BDT venue cost, but the sample outputs
// use empty seats * cost per person. This solution follows the sample outputs.
}
   if (isset($_POST["btn"])) {
calculate((int)$_POST["people"], (int)$_POST["cost"], (int)$_POST["cap"]);
}
?>

================================================================================================
Spring_2025 / Q3_PHP_MySQL_Employee_Performance
================================================================================================

--- FILE: setup.sql ---
   CREATE DATABASE IF NOT EXISTS uiutech_final;
USE uiutech_final;
DROP TABLE IF EXISTS employee_final;
   CREATE TABLE employee_final (
EmployeeID INT PRIMARY KEY,
EmployeeName VARCHAR(50) NOT NULL,
DepartmentID INT NOT NULL,
DepartmentName VARCHAR(60) NOT NULL,
Salary DECIMAL(10,2) NOT NULL,
PerformanceRating VARCHAR(2) NOT NULL
);
INSERT INTO employee_final VALUES
(1,'Arif Rahman',201,'Software Development',45000,'B'),
(2,'Marium Khan',201,'Software Development',52000,'A'),
(3,'Sabbir Hossain',202,'Quality Assurance',38000,'C'),
(4,'Samira Begum',203,'UI/UX Design',42000,'B');

--- FILE: q3.php ---
   <?php
$conn = new mysqli("localhost", "root", "", "uiutech_final");
   if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}

echo "<h3>1. Employees per performance rating</h3>";
$res = $conn->query("SELECT PerformanceRating, COUNT(*) AS total
FROM employee_final GROUP BY PerformanceRating");
   while ($row = $res->fetch_assoc()) {
echo $row["PerformanceRating"] . ": " . $row["total"] . "<br>";
}

$conn->query("UPDATE employee_final SET PerformanceRating='C'
WHERE Salary < 40000 AND PerformanceRating <> 'D'");
$conn->query("UPDATE employee_final SET Salary = Salary + 5000
WHERE Salary > 50000 AND Salary + 5000 <= 60000");
```

```
echo "<p>2. Performance ratings updated where required.</p>";
echo "<p>3. Salary bonuses updated where the new salary does not exceed 60000.</p>";

echo "<h3>4. Employees per department</h3>";
$res = $conn->query("SELECT DepartmentName, COUNT(*) AS total
FROM employee_final GROUP BY DepartmentName ORDER BY total DESC");
   while ($row = $res->fetch_assoc()) {
echo $row["DepartmentName"] . ": " . $row["total"] . "<br>";
}
$conn->close();
?>

==========================================================================================
Summer_2025 / Q1_JS_Calorie_Tracker
==========================================================================================

--- FILE: index.html ---
<!DOCTYPE html>
   <html lang="en">
<head><meta charset="UTF-8"><title>Calorie Tracker</title></head>
<body>
<input type="number" id="cal">
<button onclick="addCalories()">Add Calories</button><br>
<label id="out"></label>
<script src="script.js"></script>
</body>
</html>

--- FILE: script.js ---
   let goal = 2000;
   let total = 0;
   let entries = 0;
   function addCalories() {
       let cal = Number(document.getElementById("cal").value);
total += cal;
entries++;
       let feedback;
       if (total <= 800) { feedback = "You're off to a healthy start!"; }
else if (total <= 1600) { feedback = "Good progress, keep it balanced!"; }
else if (total < goal) { feedback = "Almost at your limit!"; }
else { feedback = "Goal reached! Stay mindful!"; }
       if (entries > 10 && total < goal) {
feedback = "Be cautious of frequent snacking!";
}

document.getElementById("out").innerText =
"Total: " + total + " / " + goal + " (" + (total / goal * 100).toFixed(1) + "%)\n" +
"Entries: " + entries + "\n" + feedback;
}
```

```
================================================================================================
Summer_2025 / Q2_PHP_Movie_Screens
================================================================================================

--- FILE: q2.php ---
<form method="post">
Attendees: <input type="number" name="people"><br>
Seat capacity: <input type="number" name="cap"><br>
Ticket price: <input type="number" name="price"><br>
<button name="btn">Calculate</button>
</form>
   <?php
   function calculate($people, $cap, $price) {
       if ($cap <= 0) {
echo "Seat capacity must be greater than zero.";
return;
}
$screens = ceil($people / $cap);
$empty = $screens * $cap - $people;
$wasted = $empty * $price;

echo "<table border='1'><tr><th>Total Screens</th><th>Empty Seats</th><th>Wasted Money</th></tr>";
echo "<tr><td>$screens</td><td>$empty</td><td>" . number_format($wasted, 2) . "</td></tr></table>";
// Fixed rental cost 25,000 BDT is not included in the paper's sample waste calculation.
}
   if (isset($_POST["btn"])) {
calculate((int)$_POST["people"], (int)$_POST["cap"], (int)$_POST["price"]);
}
?>

================================================================================================
Summer_2025 / Q3_PHP_MySQL_Sales_Data
================================================================================================

--- FILE: setup.sql ---
   CREATE DATABASE IF NOT EXISTS sundarban;
USE sundarban;
DROP TABLE IF EXISTS sales_data;
   CREATE TABLE sales_data (
SaleID INT PRIMARY KEY,
ProductName VARCHAR(50) NOT NULL,
CategoryID INT NOT NULL,
CategoryName VARCHAR(50) NOT NULL,
Quantity INT NOT NULL,
Revenue DECIMAL(12,2) NOT NULL
);
INSERT INTO sales_data VALUES
(1,'Laptop',301,'Electronics',5,350000),
(2,'Mouse',301,'Electronics',15,45000),
```

```
(3,'Chair',302,'Furniture',8,64000),
(4,'Desk',302,'Furniture',6,72000),
(5,'Bottle',303,'Accessories',20,30000),
(6,'Pen',303,'Accessories',25,20000);

--- FILE: q3.php ---
   <?php
$conn = new mysqli("localhost", "root", "", "sundarban");
   if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}

echo "<h3>1. Revenue per category (before updates)</h3>";
$res = $conn->query("SELECT CategoryName, SUM(Revenue) AS total
FROM sales_data GROUP BY CategoryName");
   while ($row = $res->fetch_assoc()) {
echo $row["CategoryName"] . ": " . $row["total"] . "<br>";
}

$conn->query("UPDATE sales_data SET CategoryName='Low Performing' WHERE Revenue < 40000");
$conn->query("UPDATE sales_data SET Revenue = Revenue * 1.10 WHERE Revenue > 70000");
echo "<p>2. Category updates completed.</p>";
echo "<p>3. Revenue bonuses completed.</p>";

// Read the average revenue for each category after Tasks 2 and 3.
$averages = array();
$res = $conn->query("SELECT CategoryName, AVG(Revenue) AS avg_revenue
FROM sales_data GROUP BY CategoryName");
   while ($row = $res->fetch_assoc()) {
$averages[$row["CategoryName"]] = $row["avg_revenue"];
}

echo "<h3>4. Seller label</h3>";
$res = $conn->query("SELECT ProductName, CategoryName, Revenue FROM sales_data");
   while ($row = $res->fetch_assoc()) {
       if ($row["Revenue"] > $averages[$row["CategoryName"]]) {
$label = "Top Seller";
} else {
$label = "Regular Seller";
}
echo $row["ProductName"] . " | " . $row["CategoryName"] . " | " . $label . "<br>";
}
$conn->close();
?>

================================================================================
Fall_2025 / Q1_PHP_CT_Marks
================================================================================

--- FILE: q1.php ---
```

```
<form method="post">
CT1: <input type="number" step="any" name="ct1"><br>
CT2: <input type="number" step="any" name="ct2"><br>
CT3: <input type="number" step="any" name="ct3"><br>
Mid: <input type="number" step="any" name="mid"><br>
Final: <input type="number" step="any" name="final"><br>
<button name="btn">Calculate</button>
</form>
   <?php
   if (isset($_POST["btn"])) {
$ct1 = (float)$_POST["ct1"];
$ct2 = (float)$_POST["ct2"];
$ct3 = (float)$_POST["ct3"];
$mid = (float)$_POST["mid"];
$final = (float)$_POST["final"];

$bestTwoAverage = ($ct1 + $ct2 + $ct3 - min($ct1, $ct2, $ct3)) / 2;
$total = $bestTwoAverage + $mid + $final;
       if ($total > 54) {
$status = "Passed";
} else {
$status = "Failed";
}

echo "Best two CT average: $bestTwoAverage<br>";
echo "Mid: $mid<br>Final: $final<br>Total: $total<br>Status: $status";
}
?>

==========================================================================================
Fall_2025 / Q2_PHP_Sales_Target
==========================================================================================

--- FILE: q2.php ---
<form method="post">
Items per day: <input type="number" name="daily"><br>
Days: <input type="number" name="days"><br>
Target: <input type="number" name="target"><br>
<button name="btn">Check</button>
</form>
   <?php
   if (isset($_POST["btn"])) {
$total = (int)$_POST["daily"] * (int)$_POST["days"];
$target = (int)$_POST["target"];

// The written rule says Excellent >= 500, but the sample labels 400 as Excellent.
// Use >= 400 here to match the sample output.
       if ($total >= 400) {
$category = "Excellent";
```

```
} elseif ($total >= 300) {
$category = "Good";
} elseif ($total >= 150) {
$category = "Average";
} else {
$category = "Poor";
}

$difference = $total - $target;
       if ($difference == 0) {
$result = "Target met exactly (0 difference)";
} elseif ($difference > 0) {
$result = "Above target by $difference items";
} else {
$result = "Below target by " . abs($difference) . " items";
}

echo "Total items sold: $total<br>Performance: $category<br>";
echo "Target: $target<br>Result: $result";
}
?>

==========================================================================================
Fall_2025 / Q3_PHP_MySQL_Library_Loans
==========================================================================================

--- FILE: setup.sql ---
   CREATE DATABASE IF NOT EXISTS campus_library;
USE campus_library;
DROP TABLE IF EXISTS book_loans;
   CREATE TABLE book_loans (
LoanID INT PRIMARY KEY,
StudentName VARCHAR(50) NOT NULL,
BookTitle VARCHAR(50) NOT NULL,
DaysOverdue INT NOT NULL,
PenaltyFee DECIMAL(10,2) NOT NULL,
Status VARCHAR(20) NOT NULL
);
INSERT INTO book_loans VALUES
(101,'Abdul','Data Structures',0,0.00,'Returned'),
(102,'Jabbar','Operating Systems',12,24.00,'Overdue'),
(103,'Barkat','Discrete Math',5,10.00,'Overdue'),
(104,'Rahim','Linear Algebra',2,4.00,'Overdue'),
(105,'Karim','Data Structures',15,30.00,'Lost'),
(106,'Fahim','Operating Systems',0,0.00,'Returned');

--- FILE: q3.php ---
   <?php
$conn = new mysqli("localhost", "root", "", "campus_library");
```

```
   if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}

echo "<h3>1. Status counts (only counts above 1)</h3>";
$res = $conn->query("SELECT Status, COUNT(*) AS total FROM book_loans
GROUP BY Status HAVING COUNT(*) > 1");
   while ($row = $res->fetch_assoc()) {
echo $row["Status"] . ": " . $row["total"] . "<br>";
}

$conn->query("UPDATE book_loans SET Status='Grace Period', PenaltyFee=0
WHERE Status='Overdue' AND DaysOverdue < 7");
$conn->query("UPDATE book_loans SET PenaltyFee = PenaltyFee * 1.10
WHERE PenaltyFee > 20 AND PenaltyFee * 1.10 <= 50");
echo "<p>2. Grace-period updates completed.</p>";
echo "<p>3. Processing charges completed where the new fee is at most 50.</p>";

echo "<h3>4. Total penalty by book</h3>";
$res = $conn->query("SELECT BookTitle, SUM(PenaltyFee) AS total
FROM book_loans GROUP BY BookTitle ORDER BY total DESC");
   while ($row = $res->fetch_assoc()) {
echo $row["BookTitle"] . ": " . number_format($row["total"], 2) . "<br>";
}
$conn->close();
?>

================================================================================================
Spring_2026 / Q1_JS_Study_Tracker
================================================================================================

--- FILE: index.html ---
<!DOCTYPE html>
   <html lang="en">
<head><meta charset="UTF-8"><title>Study Tracker</title></head>
<body>
Subject Name: <input type="text" id="subject"><br>
Study Hours: <input type="text" id="hours"><br>
<button onclick="addStudy()">Add Study</button><br>
<label id="out"></label>
<script src="script.js"></script>
</body>
</html>

--- FILE: script.js ---
   let goal = 20;
   let totalHours = 0;
   let sessions = 0;
   let subjects = [];
   function addStudy() {
       let subject = document.getElementById("subject").value;
```

```
        let hours = Number(document.getElementById("hours").value);
```

totalHours += hours;

sessions++;

subjects.push(subject);

```
        let average = totalHours / sessions;
        let feedback;
        if (totalHours <= 5) { feedback = "Start studying regularly!"; }
```

else if (totalHours <= 12) { feedback = "Good progress!"; }

else if (totalHours < goal) { feedback = "Almost ready!"; }

else { feedback = "Weekly goal reached!"; }

```
        if (sessions > 7 && totalHours < goal) {
```

feedback = "Increase your study time!";

}

document.getElementById("out").innerText =

"Total: " + totalHours + " hours, Average: " + average +

" hours, Subjects: [" + subjects.join(", ") + "], " + feedback;

}

==========

Spring_2026 / Q2_PHP_Academic_Credits

==========

--- FILE: q2.php ---

<form method="post">

Credits per course: <input type="number" step="any" name="credits"><br>

Courses completed: <input type="number" name="courses"><br>

Target credits: <input type="number" step="any" name="target"><br>

<button name="btn">Calculate</button>

</form>

```
   <?php
   if (isset($_POST["btn"])) {
```

$credits = (float)$_POST["credits"];

$courses = (int)$_POST["courses"];

$total = $credits * $courses;

$target = (float)$_POST["target"];

```
        if ($total >= 120) {
```

$progress = "Excellent";

} elseif ($total >= 90) {

$progress = "Good";

} elseif ($total >= 60) {

$progress = "Average";

} else {

$progress = "Poor";

}

$difference = $total - $target;

```
        if ($difference == 0) {
```

$result = "Target met exactly (0 difference)";

```
} elseif ($difference > 0) {
$result = "Above target by " . $difference;
} else {
$result = "Below target by " . abs($difference);
}

echo "Total credits: $total<br>Progress: $progress<br>";
echo "Target: $target<br>Result: $result";
}
?>
```

==========

Spring_2026 / Q3_PHP_MySQL_Tourism

==========

--- FILE: setup.sql ---

```
   CREATE DATABASE IF NOT EXISTS tourism;
USE tourism;
DROP TABLE IF EXISTS tourist_spot;
   CREATE TABLE tourist_spot (
SpotID INT PRIMARY KEY,
SpotName VARCHAR(50) NOT NULL,
Region VARCHAR(10) NOT NULL,
Category VARCHAR(30) NOT NULL,
Rating DECIMAL(3,1) NOT NULL,
EntryFee DECIMAL(10,2) NOT NULL,
VisitorsPerYear INT NOT NULL
);
INSERT INTO tourist_spot VALUES
(1,'Cox''s Bazar Beach','CTG','Beach',4.9,0,150000),
(2,'Sundarbans Forest','KHL','Nature',4.3,150,80000),
(3,'Ratargul Forest','SYL','Nature',4.8,100,60000),
(4,'Kuakata Beach','PTU','Beach',4.5,0,90000),
(5,'Shat Gambuj Mosque','BGH','Heritage',4.7,30,70000),
(6,'Paharpur','NWG','Heritage',4.1,30,40000);
```

--- FILE: q3.php ---

```
   <?php
$conn = new mysqli("localhost", "root", "", "tourism");
   if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error);
}

echo "<h3>1. Selected tourist spots</h3>";
$res = $conn->query("SELECT SpotName, Region, Rating FROM tourist_spot
WHERE Rating > 4.5 OR SpotName LIKE '%Beach%'
ORDER BY Rating DESC");
   while ($row = $res->fetch_assoc()) {
echo $row["SpotName"] . " | " . $row["Region"] . " | " . $row["Rating"] . "<br>";
```

```
}

$conn->query("UPDATE tourist_spot
SET Rating = Rating + 0.2, EntryFee = EntryFee * 1.10
WHERE Rating BETWEEN 4.0 AND 4.5 AND EntryFee > 0");
echo "<p>2. Updates completed.</p>";

echo "<h3>3. Statistics by category</h3>";
$res = $conn->query("SELECT Category, COUNT(*) AS spots, AVG(Rating) AS avg_rating,
SUM(VisitorsPerYear) AS visitors
FROM tourist_spot GROUP BY Category
HAVING AVG(Rating) > 4.4 ORDER BY avg_rating DESC");
   while ($row = $res->fetch_assoc()) {
echo $row["Category"] . " | " . $row["spots"] . " | " . $row["avg_rating"] .
" | " . $row["visitors"] . "<br>";
}

echo "<h3>4. Estimated yearly revenue by region</h3>";
$res = $conn->query("SELECT Region, SUM(EntryFee * VisitorsPerYear) AS revenue
FROM tourist_spot GROUP BY Region
HAVING SUM(EntryFee * VisitorsPerYear) > 1000000
ORDER BY revenue DESC");
   while ($row = $res->fetch_assoc()) {
echo $row["Region"] . " | " . number_format($row["revenue"], 2) . "<br>";
}
$conn->close();
?>
```